

Relatório Desafio Labsec

Por: Pedro Henrique Scheidt Prazeres (22100919)

Este desafio mostrou-se de grande valor formativo. As tarefas propostas proporcionaram aprendizados significativos e a oportunidade de aprofundar o contato com uma área de interesse profissional futuro, pela qual sou grato.

Cabe registrar que não foi possível concluir a totalidade das tarefas. Essa limitação decorreu, em grande parte, da escolha deliberada de priorizar a qualidade e a correteza das entregas realizadas, dedicando o tempo disponível à maior completude possível de cada implementação.

Tarefa 1

Para verificar as assinaturas, utilizei o website da união européia

<https://ec.europa.eu/digital-building-blocks/DSS/webapp-demo/validation> assim como uma análise individual de cada assinatura.

Todas as assinaturas não possuem uma Trust Anchor confiável, então todas elas são, por definição, inválidas. Contudo, dado a natureza de exercício desta atividade, esse ponto será desconsiderado na análise.

A questão pede para verificar se o conteúdo da assinatura bate com o que o nome do arquivo diz tratar, conforme as normas vigentes. Nesse caso, cada assinatura deve ser estudada individualmente.

codes_ad-rv_v2.2.p7s

O perfil AD-RV é tratado pelo ETSI EN 319 122-1, que requer as seguintes informações:

Requisito	Cades_ad_rv_v2.2.p7s
Content Type	contentType: pkcs7-signedData (1.2.840.113549.1.7.2)
Message Digest	object: messageDigest (1.2.840.113549.1.9.4) set: OCTET STRING: 0000 - 7f 98 c0 b1 ea 69 24 11-f4 e0 df c4 66i\$......f 000d - ff b8 2d 52 e0 22 ec 57-b3 64 66 b3 60 ...-R."W.df.' 001a - 6f 55 8a de 87 d7 oU....
Signing-certificate-v2 com sha-256+	cert_info: version: 2 serialNumber: 18 signature: algorithm: sha512WithRSAEncryption (1.2.840.113549.1.1.13) parameter: NULL

Signature policy identifier com OID ICP-Brasil	OBJECT SEQUENCE :2.16.76.1.7.1.3.2.2	
Signature time stamp (não assinado)	unsignedAttrs: <ABSENT> prazeres@pedro-prazer	
complete-certificate-references (não assinado)	unsignedAttrs: <ABSENT> prazeres@pedro-prazer	
complete-revocation-references (não assinado)	unsignedAttrs: <ABSENT> prazeres@pedro-prazer	
certificate-values (não assinado)	unsignedAttrs: <ABSENT> prazeres@pedro-prazer	
revocation-values (não assinado)	unsignedAttrs: <ABSENT> prazeres@pedro-prazer	

Comandos usados:

```
openssl cms -verify -in cades_ad-rv_v2.2.p7s -inform DER -noverify -out /dev/null
```

```
openssl cms -cmsout -in cades_ad-rv_v2.2.p7s -inform DER -print -noout | grep -A 20 unsignedAttrs
```

```
openssl cms -cmsout -in cades_ad-rv_v2.2.p7s -inform DER -print -noout | grep -A 20 signedAttrs
```

```
openssl asn1parse -in cades_b.p7s -inform DER -i -dump | grep -A1 OBJECT
```

```
signerInfos:  
  version: 1  
  d.issuerAndSerialNumber:  
    issuer: CN=RITE_OF_PASSAGE/serialNumber=4, C=BR, O=UFSC, OU=LabSEC, L=Florianopolis, ST=SC  
    serialNumber: 18  
  digestAlgorithm:  
    algorithm: sha256 (2.16.840.1.101.3.4.2.1)  
    parameter: <ABSENT>  
  signedAttrs:  
    object: contentType (1.2.840.113549.1.9.3)  
    set:  
      OBJECT:pkcs7-data (1.2.840.113549.1.7.1)  
  
    object: signingTime (1.2.840.113549.1.9.5)  
    set:  
      UTCTIME:Apr  8 22:15:58 2026 GMT
```

Como se pode ver, os atributos não assinados obrigatórios estão ausentes, o que torna essa assinatura INVÁLIDA

codes_b.p7s

Basic Building Blocks

SIGNATURE (Id = SIGNATURE_THE-KILLING-MOON_20260408-2215)

Format Checking : PASSED

Does the signature format correspond to an expected format? ✓

Is the signature identification not ambiguous? ✓

Is the signed references identification not ambiguous? ✓

Identification of the Signing Certificate : PASSED

Is there an identified candidate for the signing certificate? ✓

Is the signed attribute: 'cert-digest' of the certificate present? ✓

Does the certificate digest value match a digest value found in the certificate reference(s)? ✓

Are the issuer distinguished name and the serial number equal? ✓

Validation Context Initialization : PASSED

Is the signature policy known? ✓

X509 Certificate Validation : NO_CERTIFICATE_CHAIN_FOUND

Can the certificate chain be built till a trust anchor? ✗

Cryptographic Verification : PASSED

Has the reference data object been found? ✓ +

Is the reference data object intact? ✓ +

Is the signature intact? ✓

Signature Acceptance Validation : PASSED

Is the structure of the signature valid? ✓

Is the signed attribute: 'signing-certificate' present? ✓

Is the signed attribute: 'signing-certificate' present only once? !

Does the 'Signing Certificate' attribute contain references only to the certificate chain? ✓

Is the signed qualifying property: 'signing-time' present? ✓

Is the signed qualifying property: 'signing-time' within the signing-certificate's validity range? ✓

Is the signed qualifying property: 'message-digest' or 'SignedProperties' present? ✓

Are cryptographic constraints met for the signature creation? ➔ ✓ +

Algorithm Obsolescence Validation : PASSED

Are cryptographic constraints met for the signature creation? ✓ +

Are cryptographic constraints met for the signing certificate reference? ➔ ✓ +

Are cryptographic constraints met for the message digest? ✓ +

Are cryptographic constraints met for the signing certificate? ➔ ✓ +

Are cryptographic constraints met for the CA certificate? ➔ ✓ +

Are cryptographic constraints met for the CA certificate? ➔ ✓ +

Are cryptographic constraints met for the CA certificate? ➔ ✓ +

Requisito	Cades_b.p7s
Content Type	object: contentType (1.2.840.113549.1.9.3) set: OBJECT:pkcs7-data (1.2.840.113549.1.7.1)
Message Digest	object: messageDigest (1.2.840.113549.1.9.4) set: OCTET STRING: 0000 - 7f 98 c0 b1 ea 69 24 11-f4 e0 df c4 66i\$......f 000d - ff b8 2d 52 e0 22 ec 57-b3 64 66 b3 60 ...-R."W.df."oU.... 001a - 6f 55 8a de 87 d7
Signing-certificate-v2 com sha-256+	signature: algorithm: sha512WithRSAEncryption (1.2.840.113549.1.1.13) parameter: NULL

signing-time ou content-time-stamp	object: signingTime (1.2.840.113549.1.9.5) set: UTCTIME:Apr 8 22:15:58 2026 GMT
Integridade (signature value confere)	Confirmado pelo DSS
Estrutura ASN.1 bem-formada	Confirmado pelo DSS

Comandos usados:

```
openssl cms -cmsout -in cades_b.p7s -inform DER -print -noout 2>&1 | less
```

```
openssl cms -cmsout -in cades_ad-rv_v2.2.p7s -inform DER -print -noout | grep -A 20 signedAttrs
```

Por ter todos os requisitos, é VÁLIDA

jades_b.json

Basic Building Blocks
SIGNATURE (Id = SIGNATURE_THE-KILLING-MOON_20260408-2223)

Format Checking : PASSED

Does the signature format correspond to an expected format? ✓

Is the signature identification not ambiguous? ✓

Is the signed references identification not ambiguous? ✓

Identification of the Signing Certificate : PASSED

Is there an identified candidate for the signing certificate? ✓

Is the signed attribute: 'cert-digest' of the certificate present? ✓

Does the certificate digest value match a digest value found in the certificate reference(s)? ✓

Validation Context Initialization : PASSED

Is the signature policy known? ✓

X509 Certificate Validation : NO_CERTIFICATE_CHAIN_FOUND

Can the certificate chain be built till a trust anchor? ✗

Cryptographic Verification : SIGNED_DATA_NOT_FOUND

Has the reference data object been found? ✗ +

Signature Acceptance Validation : PASSED

Is the structure of the signature valid? ✓

Is the signed attribute: 'signing-certificate' present? ✓

Is the signed attribute: 'signing-certificate' present only once? ✓

Does the 'Signing Certificate' attribute contain references only to the certificate chain? ✓

Is the signed qualifying property: 'signing-time' present? ✓

Is the signed qualifying property: 'signing-time' within the signing-certificate's validity range? ✓

Are cryptographic constraints met for the signature creation? ➔ ✓ +

Algorithm Obsolescence Validation : PASSED

Are cryptographic constraints met for the signature creation? ✓ +

Are cryptographic constraints met for the signing certificate reference? ➔ ✓ +

Are cryptographic constraints met for the jws signing input? ✓ +

Are cryptographic constraints met for the signing certificate? ➔ ✓ +

Are cryptographic constraints met for the CA certificate? ➔ ✓ +

Are cryptographic constraints met for the CA certificate? ➔ ✓ +

Are cryptographic constraints met for the CA certificate? ➔ ✓ +

pades_b.pdf

Basic Building Blocks

SIGNATURE (Id = SIGNATURE_THE-KILLING-MOON_20260408-2222) 

Format Checking : PASSED	
Does the signature format correspond to an expected format?	✓
Is the signature identification not ambiguous?	✓
Is the signed references identification not ambiguous?	✓
Is only one SignerInfo present?	✓
Is the /ByteRange dictionary consistent?	✓
Does the /ByteRange not overlap with other signature/timestamp?	✓
Is the signature dictionary consistent?	✓
Do signed and final revisions contain equal amount of pages?	✓
Is no element overlapping detected in the PDF?	✓
Is there no visual difference between signed and final revisions in the PDF?	✓
Does the document contain none of the undefined object modifications?	✓
Identification of the Signing Certificate : PASSED	
Is there an identified candidate for the signing certificate?	✓
Is the signed attribute: 'cert-digest' of the certificate present?	✓
Does the certificate digest value match a digest value found in the certificate reference(s)?	✓
Are the issuer distinguished name and the serial number equal?	✓
Validation Context Initialization : PASSED	
Is the signature policy known?	✓
X509 Certificate Validation : NO_CERTIFICATE_CHAIN_FOUND	
Can the certificate chain be built till a trust anchor?	✗
Cryptographic Verification : PASSED	
Has the reference data object been found?	✓ +
Is the reference data object intact?	✓ +
Is the signature intact?	✓
Signature Acceptance Validation : PASSED	
Is the structure of the signature valid?	✓
Is the signed attribute: 'signing-certificate' present?	✓
Is the signed attribute: 'signing-certificate' present only once?	✓
Does the 'Signing Certificate' attribute contain references only to the certificate chain?	✓
Is the signed qualifying property: 'signing-time' present?	✓
Is the signed qualifying property: 'signing-time' within the signing-certificate's validity range?	✓
Is the signed qualifying property: 'message-digest' or 'SignedProperties' present?	✓
Are cryptographic constraints met for the signature creation? ➔	! +
Algorithm Obsolescence Validation : PASSED	
Are cryptographic constraints met for the signature creation?	✓ +
Are cryptographic constraints met for the signing certificate reference? ➔	! +
Are cryptographic constraints met for the message digest?	✓ +
Are cryptographic constraints met for the signing certificate? ➔	✓ +
Are cryptographic constraints met for the CA certificate? ➔	✓ +
Are cryptographic constraints met for the CA certificate? ➔	✓ +
Are cryptographic constraints met for the CA certificate? ➔	✓ +

Comando usado:

“

```
python3 -c "
```

```
import re
```

```
data = open('pades_b.pdf','rb').read()
```

```
m = re.search(rb'/Contents\s*<([0-9A-Fa-f]+)>', data)
```

```
open('embedded.p7s','wb').write(bytes.fromhex(m.group(1).decode()))  
print(f'Extraído: {len(bytes.fromhex(m.group(1).decode()))} bytes')  
"  
"
```

Para extrair a assinatura e em seguida:

```
"  
openssl asn1parse -in embedded.p7s -inform DER -i  
"
```

```
SEQUENCE  
OBJECT          :id-smime-aa-signingCertificate  
SET
```

Esta assinatura utiliza o SHA-1, algoritmo obsoleto e inseguro. Portanto é INVÁLIDO

xades_ad-rb_v2.3.xml

Basic Signing Steps	
SIGNATURE (Id = SIGNATURE_THE-KILLING-MOON)	
Format Checking : PASSED	
Does the signature format correspond to an expected format?	✓
Is the signature identification not ambiguous?	✓
Is the signed references identification not ambiguous?	✓
Identification of the Signing Certificate : PASSED	
Is there an identified candidate for the signing certificate?	✓
Is the signed attribute: 'cert-digest' of the certificate present?	✓
Does the certificate digest value match a digest value found in the certificate reference(s)?	✓
Are the issuer distinguished name and the serial number equal?	✓
Validation Context Initialization : PASSED	
Is the signature policy known?	✓
Is the signature policy available?	✓
Does the signature policy's digest value match?	!
X509 Certificate Validation : NO_CERTIFICATE_CHAIN_FOUND	
Can the certificate chain be built till a trust anchor?	✗
Cryptographic Verification : PASSED	
Has the reference data object been found?	✓ +
Is the reference data object intact?	✓ +
Has the reference data object been found?	✓ +
Is the reference data object intact?	✓ +
Is the signature intact?	✓
Signature Acceptance Validation : SIG_CONSTRAINTS_FAILURE	
Is the structure of the signature valid?	✓
Is the signed attribute: 'signing-certificate' present?	✓
Is the signed attribute: 'signing-certificate' present only once?	✓
Does the 'Signing Certificate' attribute contain references only to the certificate chain?	✓
Is the signed qualifying property: 'signing-time' present?	✗
Algorithm Obsolescence Validation : PASSED	
Are cryptographic constraints met for the signature creation?	✓ +
Are cryptographic constraints met for the signing certificate reference?	✓ +
Are cryptographic constraints met for the object reference?	✓ +
Are cryptographic constraints met for the signed properties?	✓ +
Are cryptographic constraints met for the signing certificate?	✓ +
Are cryptographic constraints met for the CA certificate?	✓ +
Are cryptographic constraints met for the CA certificate?	✓ +
Are cryptographic constraints met for the CA certificate?	✓ +

Está ausente o campo de signing time, que é obrigatório para AD-RB, como pode ser visto por meio do comando:

“

```
xmllint --xpath "boolean(//*[local-name()='SigningTime'])"
```

”

Ou verificando o xml diretamente:

```
<partly>strike</partly>
</breakfast>
<mission>-1491754572.2665806</mission>
<sentence>-1257170188</sentence>
</bring>
<earn>1483600799</earn>
<nearly>-1870164278.1732144</nearly>
<swung>-1711120069</swung>
<bicycle>1735361024</bicycle>
<accept>rock</accept>
<longer>perfectly</longer>
<molecular>forget</molecular>
</screen>
<company>atom</company>
<look>710720473.6953166</look>
<memory>-1630921579</memory>
<cry>hill</cry>
<chosen>gasoline</chosen>
<wall>-1797090212</wall>
<related>have</related>
</feel>
<mixture>running</mixture>
<time>perhaps</time>
<seldom>-1064852461.5011628</seldom>
```

(note que estas são as duas únicas menções a “time”, não existindo nem timeStamp, nem SigningTime)

xades_ad-rb_v2.5.xml

(Análise do site falhou)

Apesar do que o nome dá a entender, essa assinatura não usa AD-RB, mas na verdade AD_RC, como pode ser visto em:

```
<XAdES:SigPolicyId>
```

```
<XAdES:Identifier>2.16.76.1.7.1.9.2.5</XAdES:Identifier>
```

```
</XAdES:SigPolicyId>
```

(o penúltimo grupo inicial com ‘1’ indica que está sendo usado o AD-RB)

e

```
<XAdES:SigPolicyQualifier>
```

```
<XAdES:SPURI>http://politicas.icpbrasil.gov.br/PA_AD_RC_v2_5.xml</XAdES:SPURI>
```

```
</XAdES:SigPolicyQualifier>
```

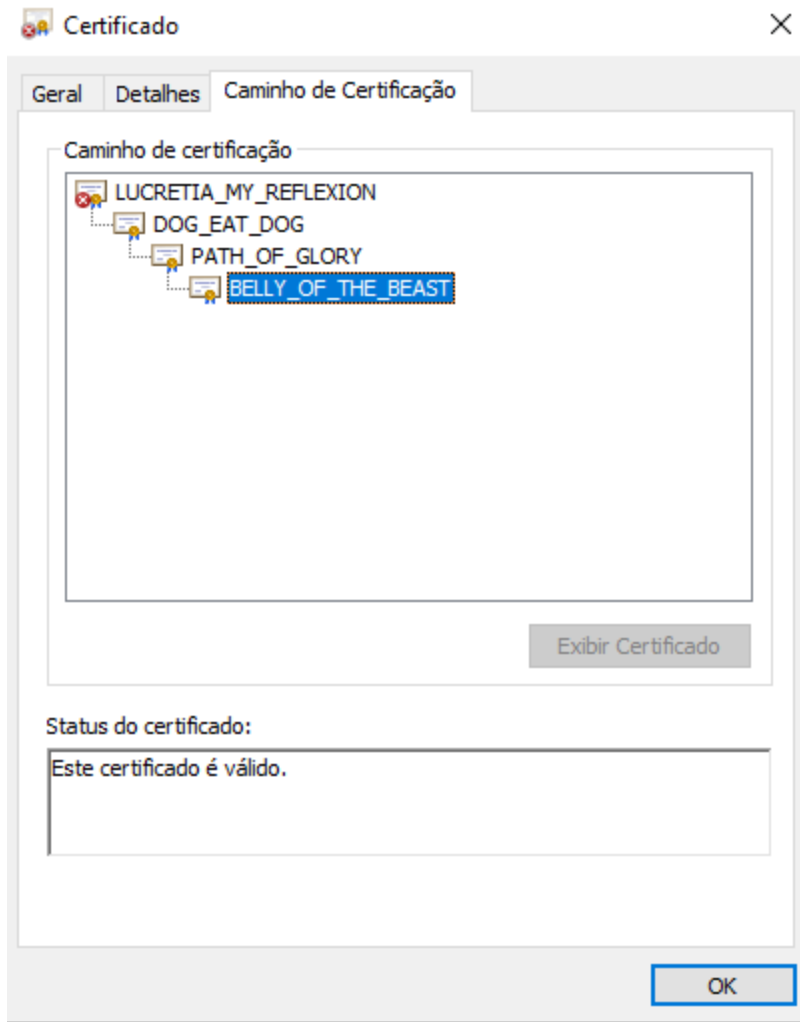
Por usar uma assinatura diferente da proposta este é INVALIDO

Tarefa 2

Rodei o comando “openssl storeutl -noout -text -certs cert_BELLY_OF_THE_BEAST.pem” e vi que o certificado acima deste se chama “PATH_OF_GLODY”, ao invés de baixar esse certificado e verificar repetir esse processo múltiplas vezes, converti o certificado para o formato .p7s com o comando

```
> openssl crl2pkcs7 -nocrl -certfile cert_BELLY_OF_THE_BEAST.pem -
out cert_BELLY_OF_THE_BEAST.pem.p7s
```

E o instalei em uma VM do windows, recebendo a seguinte trust chain:



Tarefa 5

Esta tarefa foi bem direta, apenas extrair dados de um arquivo. Foi feita por meio de leitura exhaustiva e REGEX (ctrl + f do termo “Svctype”)

Tarefa 8

Inicialmente o problema parecia paradoxal, uma vez que um terceiro ter posse da chave privada de um usuário aparentava ser contrário a princípios básicos de segurança. Após pesquisado o assunto, ilucidou-se a função de um PSC e o funcionamento de um Hardware Security Module, além do funcionamento do OAuth2, o que permitiu melhor compreensão da tarefa proposta.

Após uma exploração do código template fornecido e foram implementadas as funções sugeridas, algumas decisões notáveis:

Em model/PscUser.java e semelhantes:

```
public class PscUser {  
    @Id  
    @GeneratedValue(strategy = GenerationType.UUID)  
    @Setter(AccessLevel.NONE)  
    private String id;  
    private String name;  
    private String cpf;  
    private String defaultCertificateAlias;  
  
    @ElementCollection(fetch = FetchType.EAGER)  
    @CollectionTable(name = "user_certificates", joinColumns = @JoinColumn(name = "user_id"))  
    @Setter(AccessLevel.NONE)  
    private List<UserCertificate> certificates;  
}
```

O setter “`@Setter(AccessLevel.NONE)`” foi chamado duas vezes no arquivo template, esse e outros semelhantes foram removidos

A criação da base de dados foi feita de forma simples, devido a experiência prévia com desenvolvimento Back-End. Foram adicionados alguns novos campos e funções para os modelos, como “codeChallenge”, “scope” na definição do modelo de sessão, além dos métodos de query no arquivo de repositório.

Também foram criados dois arquivos de utilitários, “CertificateGenerator” para as funções de certificados (Criação de chaves e emissão de certificados) e “SignatureHelper” para assinaturas digitais. Essas bibliotecas foram reaproveitadas do desafio de 2025.1, que tratou pesadamente de certificados e assinaturas digitais. [O link para o relatório detalhado da criação de cada função está disponível aqui.](#)

Em um overview, deve-se destacar que foi deixada uma quantia considerável de *magic values*, isso é, valores de variáveis *hardcoded*. Embora considerada má prática, por reduzir a escalabilidade do código, devido a natureza deste desafio, elegi manter os valores *hardcoded* para tornar o código mais legível, uma vez que esse código não será expandido em uma biblioteca real, ou um projeto que será escalado. Destacar-se-a, também, que não é possível adicionar novos usuários diretamente por API, para colocar novos usuários é necessário editar o arquivo “configuration/LoadDatabase.java” e colocar mais chamadas de createUser.

Também foi disponibilizado o arquivo “userGuide.md”, que define o passo a passo para utilizar a aplicação.